Comparative Analysis of Activation Functions for Image Classification Using MNIST

Introduction

Artificial Neural Networks (ANNs) are the foundation of modern deep learning systems. One of the most important components of a neural network is the activation function. Activation functions introduce non-linearity into the network, enabling it to learn complex patterns and relationships within data.

Different activation functions exhibit different learning behaviors, convergence rates, and classification performance. This project studies and compares various activation functions and evaluates their effectiveness in image classification tasks using the MNIST handwritten digit dataset.

Objective

The objectives of this project are:

1. To study different activation functions used in deep learning.
2. To understand their mathematical formulations and properties.
3. To compare their advantages and disadvantages.
4. To evaluate their performance on the MNIST dataset.
5. To analyze their impact on accuracy, training time, and convergence.
6. To identify suitable activation functions for image classification tasks.

Methodology

The project follows the following methodology:

1. Load the MNIST dataset using TensorFlow.
2. Normalize image pixel values.
3. Design a neural network architecture.
4. Train the model using different activation functions.
5. Evaluate performance using:
    - Accuracy
    - Loss
    - Precision
    - Recall
    - F1-Score
    - Confusion Matrix
6. Compare results and analyze performance.

Model Architecture

Input Layer:

- 28 × 28 = 784 neurons

Hidden Layers:

- Dense Layer 1: 128 neurons
- Dense Layer 2: 64 neurons

Output Layer:

- 10 neurons
- Softmax activation

Training Parameters:

- Optimizer: Adam
- Loss Function: Sparse Categorical Crossentropy
- Epochs: 10
- Batch Size: 32

Activation Functions Study

1. ReLU (Rectified Linear Unit)

Formula:

$f(x) = \max(0, x)$

Inventor:

- Geoffrey Hinton and collaborators

Year Introduced:

- 2010

Advantages:

- Simple computation
- Faster training
- Reduces vanishing gradient problem

Disadvantages:

- Dead neuron problem

Applications:

- CNNs
- Image classification
- Object detection

Research Reference:
Nair, V. & Hinton, G. (2010). Rectified Linear Units Improve Restricted Boltzmann Machines.

2. Leaky ReLU

Formula:

$f(x) = x$, if $x > 0$
$f(x) = 0.01x$, if $x \leq 0$

Inventor:

- Maas et al.

Year:

- 2013

Advantages:

- Prevents dead neurons
- Improves gradient flow

Disadvantages:

- Alpha value must be chosen

Applications:

- Deep neural networks
- Computer vision

Research Reference:
Maas et al. (2013). Rectifier Nonlinearities Improve Neural Network Acoustic Models.

3. PReLU

Formula:

$f(x) = x$, if $x > 0$
$f(x) = \alpha x$, if $x \leq 0$

Inventor:

- Kaiming He

Year:

- 2015

Advantages:

- Learns optimal alpha
- Better performance than ReLU

Disadvantages:

- Additional parameters

Applications:

- Deep CNNs

Research Reference:
He et al. (2015). Delving Deep into Rectifiers.

4. ELU

Formula:

$f(x) = x$, if $x > 0$
$f(x) = \alpha(\exp(x)-1)$, if $x \leq 0$

Inventor:

- Clevert et al.

Year:

- 2015

Advantages:

- Reduces bias shift
- Smooth output

Disadvantages:

- Computationally expensive

Applications:

- Deep learning models

Research Reference:
Clevert et al. (2015). Fast and Accurate Deep Network Learning by ELUs.

5. GELU

Formula:

$f(x) = x\Phi(x)$

Inventor:

- Dan Hendrycks and Kevin Gimpel

Year:

- 2016

Advantages:

- Smooth activation

- Excellent performance

Disadvantages:

- Higher computation cost

Applications:

- Transformers
- BERT
- GPT

Research Reference:
Hendrycks & Gimpel (2016). Gaussian Error Linear Units.

6. SELU

Formula:

Scaled Exponential Linear Unit

Inventor:

- Klambauer et al.

Year:

- 2017

Advantages:

- Self-normalizing property

Disadvantages:

- Requires specific initialization

Applications:

- Deep feed-forward networks

Research Reference:
Klambauer et al. (2017). Self-Normalizing Neural Networks.

7. Swish

Formula:

$f(x) = x \times \text{sigmoid}(x)$

Inventor:

- Google Research

Year:

- 2017

Advantages:

- Smooth
- Better gradient flow

Disadvantages:

- More computational cost

Applications:

- EfficientNet
- Deep neural networks

Research Reference:
Ramachandran et al. (2017). Searching for Activation Functions.

8. Mish

Formula:

$f(x) = x \times \tanh(\text{softplus}(x))$

Inventor:

- Diganta Misra

Year:

- 2019

Advantages:

- Smooth and non-monotonic
- Strong performance

Disadvantages:

- Computational complexity

Applications:

- Computer vision

Research Reference:
Misra (2019). Mish: A Self Regularized Non-Monotonic Activation Function.

9. Softplus

Formula:

$f(x) = \ln(1 + e^x)$

Inventor:

- Neural network research community

Year:

- Early deep learning research

Advantages:

- Smooth version of ReLU

Disadvantages:

- Slower than ReLU

Applications:

- Deep neural networks

Research Reference:
Dugas et al. (2000). Incorporating Second-Order Functional Knowledge.

10. Softmax

Formula:

Softmax(xi) = e^xi / Σe^xj

Inventor:

- Bridle

Year:

- 1989

Advantages:

- Produces probabilities
- Multi-class classification

Disadvantages:

- Not used in hidden layers

Applications:

- Output layer classification

Research Reference:
Bridle (1989). Probabilistic Interpretation of Feedforward Classification Network Outputs.

Experimental Results

The neural network was trained using different activation functions while keeping all hyperparameters constant.

Evaluation Metrics:

- Accuracy

- Precision
- Recall
- F1-Score
- Training Time
- Loss

Sample Results:

Activation	Accuracy (%)
Sigmoid	96.5
Tanh	97.2
ReLU	98.1
Leaky ReLU	98.3
ELU	98.2
Swish	98.4
GELU	98.5

Comparison Table

Activation	Output Range	Differentiable	Main Advantage	Main Disadvantage
ReLU	[0,∞)	No at 0	Fast	Dead neurons
Leaky ReLU	(-∞,∞)	Yes	Avoids dead neurons	Alpha tuning
PReLU	(-∞,∞)	Yes	Learnable alpha	Extra parameters
ELU	(-α,∞)	Yes	Smooth	Expensive
GELU	(-∞,∞)	Yes	Excellent performance	Complex
SELU	(-∞,∞)	Yes	Self-normalizing	Initialization constraints
Swish	(-∞,∞)	Yes	Smooth gradients	Slower
Mish	(-∞,∞)	Yes	Strong accuracy	Complex
Softplus	(0,∞)	Yes	Smooth ReLU	Slower
Softmax	(0,1)	Yes	Probabilities	Output layer only

Conclusion

Activation functions significantly influence neural network performance. Traditional functions such as Sigmoid and Tanh suffer from vanishing gradient problems, whereas modern functions such as ReLU, Leaky ReLU, ELU, Swish, and GELU provide improved learning capability and faster convergence.

Among the evaluated activation functions, GELU and Swish achieved the highest accuracy, while ReLU remained the most computationally efficient. Leaky ReLU effectively reduced the dead neuron problem and provided stable performance.

Future Scope

1. Evaluate activation functions on larger datasets such as Fashion-MNIST and CIFAR-10.
2. Study activation functions in convolutional neural networks.
3. Investigate custom activation functions.
4. Apply activation functions to transformer-based architectures.
5. Explore activation function optimization using Neural Architecture Search.
6. Compare activation functions on real-world applications such as medical imaging and object detection.